

UNIT II

Assembly Language & Embedded Systems

Comprehensive Study Guide

Covers all topics for 15-Hour Examination

#	Topic	Key Subtopics
1	Modular Programming	Procedures, Macros, Comparison
2	Data Conversions	ASCII ↔ Hexadecimal
3	Keyboard & Video Display	DOS/BIOS Interrupts, Example Programs
4	ARM Embedded Systems	RISC, ARM Philosophy, Hardware & Software, Processor Fundamentals
5	Embedded C & Arduino Due	Arduino Board, Functions, Peripherals

Chapter 1 — Modular Programming

Modular programming is a software design technique that divides a program into separate, independent modules — each handling a distinct part of the overall task. In assembly language, this is achieved primarily through **procedures (subroutines)** and **macros**. Breaking a large program into smaller modules makes it easier to write, debug, reuse, and maintain.

1.1 Procedures (Subroutines)

A **procedure** (also called a subroutine) is a named block of assembly instructions stored once in memory that can be called from multiple places in a program. When called, execution jumps to the procedure; when the procedure finishes it returns to the instruction after the call.

How Procedures Work

When the CPU executes a **CALL** instruction, it automatically:

- Pushes the return address (address of the next instruction) onto the stack.
- Jumps to the start of the procedure.

Inside the procedure, the last instruction is **RET** (Return), which:

- Pops the return address from the stack.
- Transfers control back to the instruction after the original **CALL**.

Types of Procedures

NEAR procedure	Called from within the same code segment. Only the IP (Instruction Pointer) is saved/restored. Faster.
FAR procedure	Called from a different code segment. Both CS (Code Segment) and IP are saved/restored. Used in large programs.
PROC directive	Defines a procedure in assembly. Syntax: <code>procName PROC [NEAR/FAR] ... RET ... procName ENDP</code>

Assembly Syntax Example

```
PRINT_MSG PROC NEAR MOV AH, 09h ; DOS print string function MOV DX, OFFSET MSG ; address of the message INT 21h ; call DOS interrupt RET ; return to caller PRINT_MSG ENDP ; To call it: CALL PRINT_MSG
```

Advantages of Procedures

- Code is written and stored **only once** — saves memory.
- Can be called from **many different places** in the program.
- Promotes **code reuse** — once tested, can be trusted everywhere it is called.
- Makes programs **easier to read and debug** — each procedure has a single responsibility.
- Large programs can be split across **multiple files** using **EXTERN** and **PUBLIC** directives.

Disadvantages of Procedures

- Small **call overhead** — the CPU must **PUSH** the return address, jump, and later **POP**. In time-critical code this matters.

- Stack management required — parameters passed on stack must be carefully managed.

1.2 Macros

A **macro** is a named sequence of assembly instructions defined with a name. Unlike procedures, macros are **expanded inline** by the assembler — every time you use the macro name, the assembler substitutes the full code at that point. No CALL/RET is involved; the code is literally copy-pasted by the assembler.

Macro Definition and Use

; Macro definition PRINT MACRO msg MOV AH, 09h MOV DX, OFFSET msg INT 21h ENDM ; Using the macro — assembler replaces this with the 3 lines above PRINT HELLO_MSG PRINT BYE_MSG

Key Features of Macros

- **Parameters:** Macros can accept arguments (like function parameters) making them flexible.
- **LOCAL directive:** Use LOCAL to define labels inside macros that won't clash when the macro is used multiple times.
- **REPT / IRP / IRPC:** Repeat macros to generate repetitive code blocks automatically.
- **Nesting:** Macros can call other macros.

Advantages of Macros

- No call overhead — **faster execution** since there is no CALL/RET.
- Can accept **parameters** to customize behavior each time.
- Useful for repetitive code patterns — improves readability without the overhead of a procedure.
- Can generate code **at assembly time** (conditional assembly with IF/ENDIF inside macros).

Disadvantages of Macros

- **Code bloat:** Every use of the macro inserts a full copy of the code — program size grows.
- Harder to debug — errors inside a macro appear at every expansion site.
- No runtime flexibility — everything is decided at assembly time.

1.3 Procedure vs Macro — Detailed Comparison

Feature	Procedure	Macro
Code Storage	Stored once in memory	Copied (expanded) at every use
Execution Speed	Slightly slower (CALL/RET overhead)	Faster (no overhead, inline code)
Memory Usage	Efficient — one copy regardless of uses	Grows with each use (code bloat)
Parameters	Via registers or stack	Directly in macro definition
Debugging	Easier — single code location	Harder — errors appear at every expansion
Recursion	Supported (stack-based)	Not directly supported
Best Use	Large, reusable blocks called many times	Small, speed-critical snippets

Assembly Time	Processed at run time	Expanded at assembly time
---------------	-----------------------	---------------------------

Rule of thumb: Use a PROCEDURE when the code is long and called often (saves memory). Use a MACRO when the code is very short and speed is critical (saves time).

Chapter 2 — Data Conversions

Computers store all data as binary numbers. When communicating with humans (via keyboard or screen), data must be converted between **ASCII** (the character encoding standard) and **Hexadecimal** (a compact representation of binary). Understanding these conversions is essential for any assembly-language I/O routine.

2.1 Understanding ASCII and Hexadecimal

ASCII	American Standard Code for Information Interchange. Every character (letter, digit, symbol) has a 7-bit code stored as 1 byte. E.g., '0' = 30h, 'A' = 41h, 'a' = 61h.
Hexadecimal	Base-16 number system using digits 0–9 and letters A–F. Used in assembly because one hex digit represents exactly 4 bits, making binary values compact and readable.
Why Convert?	When a user types '9' on the keyboard, the CPU receives ASCII code 39h — not the number 9. To do arithmetic, you must convert 39h → 09h. To display a result, convert back 09h → 39h.

2.2 ASCII to Hexadecimal Conversion

The ASCII codes for digits and hex letters follow a predictable pattern:

Character	ASCII (hex)	Hex Value	Conversion
'0'	30h	0h	30h – 30h = 0h
'5'	35h	5h	35h – 30h = 5h
'9'	39h	9h	39h – 30h = 9h
'A'	41h	0Ah	41h – 37h = 0Ah
'F'	46h	0Fh	46h – 37h = 0Fh
'a'	61h	0Ah	61h – 57h = 0Ah
'f'	66h	0Fh	66h – 57h = 0Fh

Algorithm: ASCII → Hex

- Step 1: Read the ASCII character into a register (e.g., AL).
- Step 2: Check if AL is between 30h ('0') and 39h ('9').
 - If yes: subtract 30h → result is 0 to 9.
- Step 3: Otherwise check if AL is between 41h ('A') and 46h ('F').
 - If yes: subtract 37h → result is 0Ah to 0Fh.
- Step 4: Otherwise check if AL is between 61h ('a') and 66h ('f').
 - If yes: subtract 57h → result is 0Ah to 0Fh.

Assembly snippet — ASCII digit in AL → hex value in AL: `CMP AL, '9' ; Is it '0'–'9'? JBE IS_DIGIT SUB AL, 7 ; Adjust for 'A'–'F' (subtract extra 7 beyond 30h) IS_DIGIT: AND AL, 0Fh ; Mask upper nibble — gives 0–15`

2.3 Hexadecimal to ASCII Conversion

This is the reverse process — converting a hex value (0–15) back to its printable ASCII character so it can be displayed on screen.

Algorithm: Hex → ASCII

- Step 1: Load the hex value (0–15) into a register (e.g., AL).
- Step 2: Check if $AL \leq 9$.
 - If yes: add 30h → gives ASCII '0' to '9'.
- Step 3: If AL is 0Ah–0Fh: add 37h → gives ASCII 'A' to 'F'.

Assembly snippet — hex nibble in AL → ASCII character in AL: `CMP AL, 9 ; Is it 0–9? JBE ADD_30 ADD AL, 7 ;`
Extra offset to reach 'A'–'F' `ADD_30: ADD AL, 30h ; Convert to ASCII`

Converting a Full Byte (Two Hex Digits)

A single byte (e.g., 4Fh) contains two hex digits (4 and F). To display it:

- Step 1: Isolate the **high nibble**: `MOV BL, AL` then `SHR BL, 4` (shift right 4 bits to get upper digit).
- Step 2: Convert BL (high nibble) to ASCII and display.
- Step 3: Isolate the **low nibble**: `AND AL, 0Fh`.
- Step 4: Convert AL (low nibble) to ASCII and display.

■ **Important: Always process and display the HIGH nibble first, then the LOW nibble.**

Chapter 3 — Keyboard & Video Display

In assembly language on x86/DOS systems, interaction with the keyboard and screen is done through **software interrupts**. An interrupt is a signal that temporarily halts the CPU's current task and calls a special handler routine. Two layers exist: **DOS interrupts (INT 21h)** — high-level OS services, and **BIOS interrupts** — low-level hardware services.

3.1 What is a Software Interrupt?

The x86 instruction **INT n** triggers interrupt number n. The CPU:

- Saves the current CS:IP (code segment and instruction pointer) and FLAGS on the stack.
- Looks up the address of the interrupt service routine (ISR) in the Interrupt Vector Table (IVT) at memory address 0000:0000.
- Jumps to and executes the ISR.
- Returns with IRET, restoring CS:IP and FLAGS.

Before calling an interrupt, you load the **function number** into register AH and any required parameters into other registers (AL, BX, CX, DX, DS:DX, etc.).

3.2 DOS Interrupts — INT 21h

INT 21h is the main DOS interrupt, providing over 100 functions for file I/O, string I/O, memory management, and more. The function number is placed in **AH**.

AH Value	Function	Input	Output / Notes
01h	Read character with echo	None	AL = ASCII code of key pressed. Key is echoed (shown) on screen.
02h	Display character	DL = ASCII char to display	Displays the character in DL on screen at cursor position.
07h	Read character (no echo)	None	AL = ASCII code. Key NOT shown on screen. Good for passwords.
08h	Read char, check Ctrl-C	None	AL = ASCII code. Like 01h but no echo and Ctrl-C checked.
09h	Display string	DS:DX = address of string ending with '\$'	Prints the entire string up to the '\$' character.
0Ah	Buffered keyboard input	DS:DX = address of input buffer	Reads a full line. Byte 0 = max chars, Byte 1 = actual chars read.
0Bh	Check keyboard status	None	AL = FFh if key waiting, 00h if not. Non-blocking check.

4Ch	Terminate program (Exit)	AL = exit code (usually 0)	Ends the program and returns to DOS. Always call at program end.
-----	--------------------------	----------------------------	--

Example: Read a Key and Display It

```
MOV AH, 01h ; function: read key with echo INT 21h ; call DOS — AL now contains the ASCII code
MOV DL, AL ; move the character to DL
MOV AH, 02h ; function: display character INT 21h ; display it again
```

Example: Display a String

```
DATA SEGMENT MSG DB 'Hello, World!$' ; $ marks the end of string
DATA ENDS
CODE SEGMENT
MOV AH, 09h
MOV DX, OFFSET MSG ; DS must already point to DATA segment
INT 21h ; prints: Hello, World!
CODE ENDS
```

3.3 BIOS Interrupts

BIOS (Basic Input/Output System) interrupts are firmware-level routines stored in ROM. They operate below DOS, giving direct access to hardware. Useful when DOS is not available (e.g., in boot sector code or real-time systems).

Interrupt	Purpose	Key Functions
INT 10h	Video Services	AH=00h: Set video mode. AH=02h: Set cursor position. AH=06h: Scroll window. AH=09h: Write char+attribute. AH=0Eh: Teletype output (write char and advance cursor).
INT 16h	Keyboard Services	AH=00h: Read keystroke (blocks until key pressed). AH=01h: Check keystroke (non-blocking). AH=02h: Read shift flags (Shift, Ctrl, Alt status).
INT 14h	Serial Port (COM)	AH=00h: Initialize port. AH=01h: Send char. AH=02h: Receive char. AH=03h: Get port status.
INT 17h	Printer (LPT)	AH=00h: Print char. AH=01h: Initialize. AH=02h: Get status.

Example: Write Character Using BIOS (INT 10h Teletype)

```
MOV AH, 0Eh ; BIOS teletype output function
MOV AL, 'A' ; character to display
MOV BH, 0 ; page number (0 = active page)
INT 10h ; display 'A' on screen
```

Example: Read a Key Using BIOS (INT 16h)

```
MOV AH, 00h ; BIOS read keystroke
INT 16h ; wait for key press ; After: AL = ASCII code, AH = scan code
```

Scan Code	Hardware code sent by keyboard for every key, regardless of character. E.g., F1 has scan code 3Bh but no ASCII code.
------------------	--

ASCII Code

Character code for printable keys. Non-printable keys (arrows, F-keys) return ASCII 00h; read AH for the scan code instead.

Chapter 4 — ARM Embedded Systems

ARM (Advanced RISC Machines) is the world's most widely used processor architecture, found in virtually every smartphone, tablet, microcontroller, and embedded device. Understanding ARM's design philosophy is fundamental to modern embedded systems development.

4.1 RISC Design Philosophy

RISC — Reduced Instruction Set Computer — is a processor design philosophy based on the observation that a small set of simple instructions, each executing in one clock cycle, is more efficient than a large set of complex variable-length instructions.

Core RISC Principles

Fixed instruction size	All instructions are the same length (32 bits on ARM). This simplifies fetch and decode stages of the pipeline.
Load/Store architecture	Only LDR (load) and STR (store) instructions access memory. All other operations (ADD, SUB, AND...) work only on registers.
Large register file	RISC processors provide many general-purpose registers (ARM has 16) to minimize memory accesses.
Single-cycle execution	Most instructions complete in one clock cycle, enabling efficient pipelining.
Hardwired control	Instructions decoded directly in hardware (not microcode), resulting in fast, predictable execution.
Compiler-friendly	Simple, regular instruction set is easier for compilers to optimize than complex CISC sets.

CISC vs RISC: Intel x86 is CISC (Complex Instruction Set). It has hundreds of instructions of varying lengths. ARM is RISC: ~100 instructions, all 32 bits wide. ARM typically uses fewer transistors and less power than equivalent CISC chips.

4.2 ARM Design Philosophy

Beyond pure RISC principles, ARM Ltd. added several unique features that distinguish ARM from other RISC architectures:

Conditional Execution

Every ARM instruction can be conditionally executed based on the processor status flags (N, Z, C, V). The condition is encoded in the top 4 bits of every instruction.

- Instead of: CMP R0, R1 / BNE skip / ADD R2, R3, R4 / skip:
- ARM allows: CMP R0, R1 / ADDEQ R2, R3, R4 (ADD only executes if equal)
- Benefit: Avoids branch instructions → no pipeline flush → faster code.

Barrel Shifter

Every data-processing instruction can include a free shift/rotate of one operand. E.g., ADD R0, R1, R2, LSL #3 adds $R1 + (R2 \text{ shifted left } 3)$ in a single instruction.

Thumb Instruction Set

ARM processors support a 16-bit compressed instruction encoding called Thumb. Thumb code is ~30% smaller than ARM code with only ~5–10% performance penalty. Ideal for memory-constrained embedded systems.

ARM Processor Families

Family	Architecture	Target
Cortex-M	ARMv6-M / ARMv7-M / ARMv8-M	Microcontrollers (Arduino Due uses Cortex-M3)
Cortex-R	ARMv7-R / ARMv8-R	Real-time systems (automotive, industrial)
Cortex-A	ARMv7-A / ARMv8-A (64-bit)	Application processors (smartphones, tablets)
Classic ARM	ARM7, ARM9, ARM11	Legacy embedded systems

4.3 Embedded System Hardware

An **embedded system** is a computer system designed for a specific function within a larger device — unlike a general-purpose computer. Examples: washing machine controller, car ABS brakes, medical pacemaker, router.

Key Hardware Components

Microcontroller (MCU)	Single chip containing CPU + RAM + Flash memory + I/O peripherals. The 'brain' of the embedded system. E.g., Arduino Due uses Atmel SAM3X8E (ARM Cortex-M3 @ 84 MHz).
Memory types	Flash ROM: stores the program (non-volatile, survives power off). SRAM: stores variables during execution (volatile, lost on power off). EEPROM: non-volatile user data storage.
GPIO (General Purpose I/O)	Digital pins that can be configured as INPUT or OUTPUT. Used to read sensors (switches, encoders) or control devices (LEDs, motors).
ADC (Analog-to-Digital)	Converts a continuous analog voltage (e.g., 0–3.3V from a sensor) into a digital number (0–4095 on Arduino Due's 12-bit ADC).
DAC (Digital-to-Analog)	Converts a digital value back to an analog voltage. Arduino Due has 2 DAC outputs for audio or analog control signals.
Timers / Counters	Hardware modules that count clock cycles independently of the CPU. Used for generating precise delays, PWM signals, measuring pulse widths.
UART / SPI / I2C	Communication peripherals for talking to other chips. UART = serial (USB/Bluetooth). SPI = fast short-distance chip-to-chip. I2C = multi-device bus.
Interrupts	Hardware signals that immediately pause the CPU and call an ISR (Interrupt Service Routine). Essential for real-time response (e.g., button press).

Memory-Mapped I/O: In ARM systems, every peripheral register has a unique memory address. Writing to that address configures or controls the peripheral. E.g., writing 1 to address 0x400FF084 might turn on GPIO pin 5.

4.4 Embedded System Software

Embedded software runs in resource-constrained environments — limited RAM, no hard drive, often no operating system. Two main approaches:

Bare-Metal Programming

The program runs directly on hardware with no operating system. The programmer has full control over every hardware register. All timing, I/O, and task management is done manually.

- Advantages: Maximum performance, minimum memory use, fully deterministic (predictable timing).
- Disadvantages: Complex to write and maintain, no built-in scheduling or file system.
- Used in: Safety-critical systems, ultra-low-power devices, simple single-task applications.

RTOS (Real-Time Operating System)

A lightweight OS designed for embedded systems, providing task scheduling, inter-task communication, and timing services. Examples: FreeRTOS, Zephyr, VxWorks.

- Preemptive scheduler: Can interrupt a lower-priority task to run a higher-priority one.
- Tasks: Independent functions that run as if they have their own CPU.
- Semaphores/Mutexes: Protect shared resources from simultaneous access by multiple tasks.
- Queues/Mailboxes: Pass data safely between tasks.

4.5 ARM Processor Fundamentals

Registers (ARM Cortex-M)

ARM has 16 general-purpose 32-bit registers plus special-purpose registers:

Register	Name	Purpose
R0–R3	Argument / Result registers	Pass function arguments and return values. Also used as scratch registers.
R4–R11	Saved registers	Must be preserved across function calls. Callee saves and restores them.
R12 (IP)	Intra-Procedure scratch	Scratch register for linker stubs and compiler use.
R13 (SP)	Stack Pointer	Points to the top of the current stack. Used by PUSH/POP, function calls.
R14 (LR)	Link Register	Automatically stores the return address when BL (Branch with Link) is called.
R15 (PC)	Program Counter	Address of the instruction being fetched. Always 2 instructions ahead (pipeline effect).

Program Status Register (PSR/CPSR/APSR)

The 32-bit status register holds condition flags set by arithmetic/logic instructions. These flags control conditional execution of subsequent instructions.

N — Negative	Set to 1 if the result of the last operation was negative (bit 31 = 1).
---------------------	---

Z — Zero	Set to 1 if the result was zero. Used after CMP to test equality.
C — Carry	Set to 1 if the operation produced a carry out (unsigned overflow).
V — Overflow	Set to 1 if the operation produced a signed overflow.
T — Thumb bit	1 = executing in Thumb (16-bit) mode. 0 = ARM (32-bit) mode.
I, F bits	Interrupt enable flags. I=1 disables IRQ, F=1 disables FIQ (fast interrupt).

Example: CMP R0, R1 subtracts R1 from R0 but discards the result — only updates flags. If R0 = R1 → Z=1 (zero result) If R0 > R1 (unsigned) → C=1 If R0 < R1 → N=1 (negative result) Then: BEQ label (Branch if Equal) jumps if Z=1.

Pipeline

The ARM processor uses a **3-stage pipeline** (more stages in Cortex-A) to overlap the processing of multiple instructions, dramatically increasing throughput.

Stage 1: Fetch	Reads the instruction from memory (Flash/RAM) at the address in PC. PC is incremented to the next instruction.
Stage 2: Decode	The instruction bits are interpreted. The control unit determines what operation to perform and which registers to use.
Stage 3: Execute	The ALU (Arithmetic Logic Unit) performs the operation. Results are written back to registers or memory.
PC offset (+8)	Because of the 3-stage pipeline, PC always points 2 instructions ahead of the one currently executing. Important for self-modifying code and branch calculations.
Branch penalty	A taken branch flushes the pipeline (the instructions already fetched are discarded), causing a 1–2 cycle penalty. Conditional execution avoids this.
Cortex-M3	Uses a 3-stage pipeline with branch speculation to minimize penalty. Arduino Due's SAM3X8E uses Cortex-M3.

Chapter 5 — Embedded C Programming & Arduino Due

The **Arduino Due** is an open-source microcontroller board based on the **Atmel SAM3X8E ARM Cortex-M3** processor running at 84 MHz. It is the first Arduino board to feature a 32-bit ARM processor, making it significantly more powerful than earlier 8-bit AVR-based Arduino boards. Programming is done in **Embedded C / C++** using the Arduino IDE.

5.1 Arduino Due Board Specifications

Processor	Atmel SAM3X8E ARM Cortex-M3 at 84 MHz
Operating voltage	3.3V (NOT 5V — connecting 5V signals will damage the board!)
Flash memory	512 KB (program storage)
SRAM	96 KB (for variables at runtime)
Digital I/O pins	54 pins (12 can provide PWM output)
Analog input pins	12 pins (12-bit ADC → values 0–4095)
Analog output (DAC)	2 pins (true analog output, not PWM)
UART ports	4 hardware serial ports (Serial, Serial1, Serial2, Serial3)
USB	2 USB ports: Programming port + Native USB port
Clock speed	84 MHz (much faster than Uno's 16 MHz)

5.2 Pinout Diagram — Key Pins

Pin Type	Pins	Description
Digital I/O	D0–D53	Digital GPIO. D0/D1 = Serial RX/TX. D2–D13 = also support interrupts on some.
PWM Output	D2–D13 (select)	Pulse Width Modulation for motor speed, LED dimming. <code>analogWrite()</code> used here.
Analog Input	A0–A11	12-bit ADC inputs. Read with <code>analogRead()</code> → 0 to 4095.
DAC Output	DAC0, DAC1	True analog output. <code>analogWrite()</code> on these pins gives genuine voltage, not PWM.
Power pins	3.3V, 5V, GND, Vin	3.3V and 5V supply rails. Vin accepts 7–12V external supply.
SPI	D10 (SS), D11 (MOSI), D12 (MISO), D13 (SCK)	Also available on the SPI header at center of board (faster access).
I2C	SDA (D20), SCL (D21)	I2C bus for sensors, displays, EEPROMs.

UART	D0/D1 (Serial), D19/D18 (Serial1)...	Hardware serial communication.
Reset	RESET pin, ERASE pin	RESET restarts the program. ERASE erases the Flash — use with caution.

5.3 Program Structure

Every Arduino program (called a **sketch**) must contain exactly two functions:

setup()	Called once when the board powers on or is reset. Used to initialize pins (pinMode), start serial communication (Serial.begin), configure libraries (lcd.begin), etc.
loop()	Called repeatedly in an infinite loop after setup() finishes. Contains the main program logic — reading sensors, updating outputs, checking conditions.

```
void setup() { pinMode(13, OUTPUT); // set pin 13 as output Serial.begin(9600); // start serial at 9600 baud }
void loop() { digitalWrite(13, HIGH); // turn LED on delay(1000); // wait 1 second digitalWrite(13, LOW); // turn LED off delay(1000); // wait 1 second }
```

5.4 Built-in Functions

pinMode(pin, mode)

Configures a specific pin to behave as INPUT, OUTPUT, or INPUT_PULLUP. Must be called in setup() before the pin can be used.

- OUTPUT: Pin can source/sink current to control LEDs, transistors, etc.
- INPUT: Pin reads external voltage (HIGH=3.3V, LOW=0V). High-impedance state.
- INPUT_PULLUP: Same as INPUT but internal pull-up resistor enabled. Pin reads HIGH when nothing connected, LOW when connected to GND.

Digital I/O

digitalWrite(pin, val)	Sets a digital OUTPUT pin HIGH (3.3V) or LOW (0V). Example: digitalWrite(13, HIGH) turns on the onboard LED.
digitalRead(pin)	Reads the state of an INPUT pin. Returns HIGH (1) or LOW (0). Example: int state = digitalRead(2);
Electrical limits	Each GPIO pin can source or sink max 8 mA on Due (less than Uno's 40 mA). Always use a current-limiting resistor with LEDs (minimum ~150Ω for 3.3V).

Analog I/O

analogRead(pin)	Reads the voltage on an analog input pin (A0–A11). Returns 0–4095 (12-bit on Due). 0 = 0V, 4095 = 3.3V. Formula: voltage = reading × (3.3 / 4095).
analogWrite(pin, val)	On PWM pins: generates a PWM signal. Value 0–255 sets duty cycle (0%–100%). On DAC0/DAC1: generates true analog voltage 0–3.3V.
analogReadResolution(bits)	Set ADC resolution. Default 10 (0–1023). Set to 12 for full 12-bit (0–4095) on Due.
analogWriteResolution(bits)	Set DAC/PWM resolution. Default 8. Can set to 12 for finer DAC control on Due.

Delay and Timing

delay(ms)	Pauses the program for the specified number of milliseconds. The CPU does nothing else during this time (blocking).
delayMicroseconds(us)	Pauses for microseconds. Useful for precise timing in communication protocols or sensor pulses.
millis()	Returns the number of milliseconds since the board started. Used for non-blocking timing (replacing delay in responsive code).
micros()	Returns microseconds since start. Wraps around (overflows) after ~70 minutes.
Non-blocking timing	Instead of delay(), compare millis() to a stored time. Allows the loop() to keep running while waiting: <code>if (millis() - prevTime >= interval) { ... }</code>

Serial Communication

Serial.begin(baud)	Initialize serial communication at the given baud rate. Common rates: 9600, 115200. Must be called in setup().
Serial.print(val)	Prints val to the serial port (USB). Does NOT add newline. Accepts int, float, String, char.
Serial.println(val)	Like Serial.print() but adds newline (\r\n) at end. Use for readable output line by line.
Serial.read()	Returns the next byte from the receive buffer (as int). Returns -1 if nothing available.
Serial.available()	Returns the number of bytes waiting in the receive buffer. Check before calling Serial.read() to avoid reading -1.
Serial.write(val)	Sends a raw byte. Unlike print(), sends the binary value, not the ASCII representation.

```
Reading from Serial safely: if (Serial.available() > 0) { char c = Serial.read(); // read one byte
Serial.print('Got: ');
Serial.println(c); }
```

5.5 Interfacing Peripherals

Below are all the peripherals listed in the syllabus, with wiring and code for each.

LED — Light Emitting Diode

An LED emits light when current flows through it. Always use a current-limiting resistor in series. For 3.3V Arduino Due: $R = (3.3V - 2.0V) / 0.02A \approx 65\Omega \rightarrow$ use 100 Ω or 220 Ω .

- Wiring: LED anode $\rightarrow$ resistor $\rightarrow$ GPIO pin. LED cathode $\rightarrow$ GND.

```
pinMode(9, OUTPUT); digitalWrite(9, HIGH); // LED on
delay(500); digitalWrite(9, LOW); // LED off
```

Switches / Push Buttons

A push button connects a pin to either GND or VCC when pressed. Use INPUT_PULLUP so the pin reads HIGH normally and LOW when button pressed.

- Wiring: One side of button $\rightarrow$ GPIO pin, other side $\rightarrow$ GND.

```
pinMode(7, INPUT_PULLUP); // internal pull-up resistor
int state = digitalRead(7); // HIGH = not pressed, LOW = pressed
if (state == LOW) { Serial.println("Button pressed!"); }
```


Buzzer

A passive buzzer generates sound when driven with a PWM signal. `tone()` automatically generates the required square wave.

- Active buzzer: just needs HIGH/LOW. Passive buzzer: needs `tone()` for different frequencies.

```
tone(8, 1000); // play 1000 Hz on pin 8 delay(500); noTone(8); // stop the sound // Or for active buzzer:
digitalWrite(8, HIGH); // buzzer on delay(200); digitalWrite(8, LOW); // buzzer off
```

Potentiometer

A potentiometer is a variable resistor with 3 terminals. The middle (wiper) outputs a voltage between 0V and VCC depending on the knob position.

- Wiring: Pin 1 → 3.3V, Pin 3 → GND, Pin 2 (wiper) → Analog pin A0.

```
int raw = analogRead(A0); // 0–4095 float voltage = raw * (3.3 / 4095.0); // convert to volts int mapped = map(raw, 0,
4095, 0, 100); // map to 0–100% Serial.println(voltage);
```

16×2 LCD Display

A 16-column by 2-row character LCD. Uses the HD44780 controller. Typically connected in 4-bit mode (uses 6 GPIO pins) via the LiquidCrystal library.

- Pins: RS, EN, D4, D5, D6, D7. Also VCC, GND, V0 (contrast — connect to potentiometer).

```
#include LiquidCrystal lcd(12, 11, 5, 4, 3, 2); // RS, EN, D4, D5, D6, D7 void setup() { lcd.begin(16, 2); // 16 cols, 2
rows lcd.print("Hello World!"); // top-left lcd.setCursor(0, 1); // move to row 2 lcd.print("Arduino Due"); // print on row 2
}
```

Seven-Segment Display

Displays a single digit (0–9) or some letters using 7 LED segments (a–g) plus decimal point. Common cathode: all cathodes share GND, drive segments HIGH. Common anode: all anodes share VCC, drive segments LOW.

```
// Segment pins: a=2, b=3, c=4, d=5, e=6, f=7, g=8 // Digit '0': segments a,b,c,d,e,f ON; g OFF byte segments[10] =
{ // a b c d e f g (bit pattern) 0b0111111, // 0 0b0000110, // 1 0b1011011, // 2 ... etc }; // Write each bit to
corresponding pin
```

Stepper Motor

A stepper motor rotates in precise discrete steps. The Stepper library drives it using the L293D or ULN2003 driver IC. A 28BYJ-48 motor has 2048 steps per revolution.

```
#include const int STEPS = 2048; // steps per revolution Stepper motor(STEPS, 8, 10, 9, 11); // IN1, IN3, IN2, IN4
pins void setup() { motor.setSpeed(10); // 10 RPM } void loop() { motor.step(2048); // one full revolution CW
delay(500); motor.step(-2048); // one full revolution CCW }
```

4×4 Matrix Keypad

A 4×4 keypad has 16 keys arranged in a 4-row × 4-column grid, using only 8 pins. The Keypad library handles scanning. The library determines which key is pressed by driving each row LOW and checking which column reads LOW.

```
#include const byte ROWS = 4, COLS = 4; char keys[4][4] = { {'1','2','3','A'}, {'4','5','6','B'}, {'7','8','9','C'}, {'*','0','#','D'}
}; byte rowPins[4] = {9,8,7,6}; // row pins byte colPins[4] = {5,4,3,2}; // column pins Keypad kpd =
Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS); void loop() { char key = kpd.getKey(); if (key) {
Serial.println(key); } }
```

Quick Summary — All Topics

Topic	Key Concepts to Remember
Modular Programming	Procedure = stored once, CALL/RET, saves memory. Macro = inline expansion, no overhead, more code size. Use procedure for large reusable code; macro for tiny fast snippets.
Data Conversions	ASCII '0'-'9': subtract 30h. ASCII 'A'-'F': subtract 37h. ASCII 'a'-'f': subtract 57h. Reverse: add 30h (0-9) or 37h (A-F).
DOS INT 21h	AH=01h read key, AH=02h display char, AH=09h print string (ends '\$'), AH=0Ah buffered input, AH=4Ch exit program.
BIOS Interrupts	INT 10h = video (AH=0Eh teletype write). INT 16h = keyboard (AH=00h read key, AH=01h check key).
RISC Principles	Fixed 32-bit instructions. Load/Store only for memory. Many registers. Single-cycle execution. Compiler-friendly.
ARM Features	Conditional execution on every instruction (avoids branches). Barrel shifter (free shift in ALU ops). Thumb = 16-bit compressed encoding.
ARM Registers	R0-R3 = args/results. R4-R11 = saved. R12=IP. R13=SP. R14=LR (return addr). R15=PC (always 2 ahead). APSR: N Z C V flags.
ARM Pipeline	3 stages: Fetch → Decode → Execute. PC is 2 instructions ahead. Branch penalty = pipeline flush. Conditional execution avoids this.
Arduino Due	SAM3X8E Cortex-M3 @ 84 MHz. 3.3V! 512KB Flash, 96KB SRAM. 12-bit ADC (0-4095). 2 DAC pins. 54 digital pins, 12 analog.
Arduino Functions	pinMode / digitalWrite / digitalRead / analogRead / analogWrite / delay / millis / Serial.begin / Serial.print / Serial.read
Peripherals	LED: resistor+digitalWrite. Switch: INPUT_PULLUP+digitalRead. Buzzer: tone(). Pot: analogRead+map. LCD: LiquidCrystal lib. Stepper: Stepper lib. Keypad: Keypad lib.